

FASTAPI · REACT · MYSQL · DOCKER

BiblioTech

A development and learning guide: what this project is, how it was built, the concepts behind every major decision, and a primer on Docker written for someone building something like this for the first time.

Repository: github.com/Daniel-Pino-2000/BiblioTech

Generated: August 2026

Prepared by Claude, working alongside the project's author

Contents

-
- [1 What BiblioTech is, and why it exists](#)
 - [2 How it was actually built — the development timeline](#)
-

[3](#) Architecture: how a request flows through the system

[4](#) REST APIs, explained through this project

[5](#) Authentication and authorization — and the difference between them

[6](#) Databases, ORMs, and migrations

[7](#) Automated testing — why, and how this project does it

[8](#) Docker, from zero: what it is and how this project uses it

[9](#) Continuous integration (CI)

[10](#) The frontend: how it talks to the backend

[11](#) Mistakes made and fixed along the way — real lessons

[12](#) A playbook for building your own version of this

[13](#) Appendix: commands, tech stack, folder structure

1. What BiblioTech is, and why it exists

BiblioTech is a full-stack bookstore application for technical books. A user can browse and search a catalog, read and leave ratings/comments, keep a shopping cart, maintain wishlists, and manage their profile. An admin account can add new books and run publisher-wide discounts. Under the hood it's a FastAPI REST API backed by MySQL, with a React/TypeScript single-page app as the frontend.

It started as a course project for **CEN 4010 — Software Engineering**, built around a Scrum-style feature breakdown (a router, service, and set of models per domain: users, books, authors, ratings, wishlist, cart). That version worked, but it had the shape of a class assignment: no authentication, no tests, a schema created by a single `create_all()` call, and at least one outright bug that made an endpoint crash on every call.

The purpose of the work documented in this guide was to take that assignment and turn it into something that demonstrates real engineering judgment — the kind of project a hiring manager can open, read, run, and trust. That meant closing every gap between "it works on my machine, once, if nothing goes wrong" and "this is how you'd actually build and ship a backend service."

Why this matters for a portfolio

A CRUD app that works is table stakes. What actually differentiates a project is whether it shows you understand *why* a real system needs authentication, authorization, migrations, tests, and CI — not just that you can copy a tutorial. Every section in this guide ties a concept back to a specific, real decision made in this codebase.

2. How it was actually built — the development timeline

This project wasn't built in one pass. It went through the same phases most real backend work goes through: get something running, find out what's actually broken, harden it, prove it works, then make it deployable. Here's the real order of operations, because the order matters as much as the individual steps — you can't meaningfully write tests before you have authentication to test, and you can't write a useful CI pipeline before you have tests for it to run.

Phase	What happened	Why this order
1. Audit	Read every router, service, model, and schema. Found a broken endpoint (book creation crashed on every call — it inserted a Pydantic object instead of a database row), a wrong-column bug in cart removal, and a <code>requirements.txt</code> saved in the wrong text encoding so a fresh install couldn't even start the app.	You can't harden code you haven't actually read. Bugs found here would have undermined every later step.
2. Fix bugs	Fixed the three issues above before touching anything else.	No point building auth on top of an endpoint that doesn't work.
3. Authentication	Added password hashing (bcrypt), JWT login, and the two auth dependencies (<code>get_current_user</code> , <code>get_current_admin_user</code>) every protected route now uses.	Nothing else — authorization, tests, a real admin flow — is possible without this existing first.
4. Authorization	Went endpoint by endpoint and added ownership checks: a user can only read/modify <i>their own</i> cart, wishlist, and profile. Catalog writes became admin-only.	Authentication answers "who are you?" — authorization answers "are you allowed to do this?" A project with one but not the other is still insecure.

5. Security hardening	Stopped storing full credit card numbers (last 4 digits only).	A small, concrete example of "secure by design" rather than "secure by accident."
6. Fill API gaps	Added a proper GET <code>/books</code> search/pagination endpoint (there was previously no way to browse the whole catalog) and a book-by-id lookup.	These gaps only became obvious once building a frontend required them.
7. Migrations	Replaced <code>Base.metadata.create_all()</code> with Alembic — a real, versioned migration history.	Needed a stable schema (everything above had to land first) before generating the initial migration.
8. Tests	27 pytest tests covering auth, cross-user authorization boundaries, and every domain.	Tests are only meaningful once there's real behavior (auth, ownership checks) worth protecting against regression.
9. Docker + CI	Dockerfile, docker-compose (API + MySQL), and a GitHub Actions workflow running the test suite and validating migrations on every push.	CI needs tests to run; Docker needs a stable app to containerize.
10. Frontend	A React/TypeScript SPA: catalog, book detail, cart, wishlist, auth, profile, and later an admin page.	Building the UI against a half-finished API would have meant redoing frontend work every time the API changed underneath it.

11. README, in-code docstrings, this guide.
Documentation

Documentation describes a finished, working thing — writing it earlier means rewriting it later anyway.

A note on commit history

Every change above was committed separately and in this order — 16+ small, single-purpose commits rather than one enormous diff. That's not cosmetic: a reviewer (or you, six months from now) can look at `git log` and understand the story of the project without reading a single line of code. If you build something similar, commit as you go, in logical units — don't wait until the end and dump everything into one commit.

3. Architecture: how a request flows through the system

The backend follows a layered architecture. Each layer has exactly one job, which makes the code easier to test and easier to change without breaking something unrelated:

Layer	Folder	Responsibility
Router	<code>app/api/routers/</code>	HTTP only: parse the request, call a dependency (auth check, DB session), call the service layer, shape the response. No business logic and no direct database queries live here.
Service	<code>app/services/</code>	Business logic and database queries. "Can a user have more than 3 wishlists?" is answered here, not in the router.
Model	<code>app/models/</code>	SQLAlchemy ORM classes — one per database table, plus the relationships between them.
Schema	<code>app/schemas/</code>	Pydantic classes defining exactly what shape of JSON goes in and comes out of each endpoint. This is also what generates the interactive docs at <code>/docs</code> .

Walking through one concrete example end to end — a logged-in user adding a book to their cart (`POST /cart/add?user_id=3&book_id=7`):

1. **FastAPI** matches the route in `app/api/routers/cart.py` and resolves its dependencies first: `get_db` opens a database session, `get_current_user` decodes the JWT bearer token into a `User` row.
2. The route function calls `_ensure_self(user_id, current_user)` — a small local check that 403s if the token's user doesn't match the `user_id` in the URL. This is the authorization step: being logged in isn't enough, you have to be *this specific user*.
3. It calls `cart_service.add_item_to_cart(db, user_id, book_id)`, which finds or creates the user's cart, checks whether the book is already in it, and either bumps the quantity or inserts a new

`CartItem` row.

4. The router returns a small JSON confirmation. FastAPI serializes it automatically based on the endpoint's return type.

The frontend mirrors this layering: `frontend/src/api/*.ts` are thin wrappers around `fetch` (one file per backend resource), `context/AuthContext.tsx` holds session state, and `pages/*.tsx` compose the two into actual screens.

4. REST APIs, explained through this project

"REST" is a set of conventions for designing an HTTP API so that its shape is predictable. The core idea: URLs identify *resources* (a book, a user, a cart), and HTTP methods describe the *action* on that resource.

Method	Example in this API	Meaning
GET	GET /books/{isbn}	Read one resource. Never changes anything.
GET	GET /books?search=clean	Read a collection, optionally filtered.
POST	POST /books/create-book	Create a new resource.
PATCH	PATCH /users/{username}	Partially update an existing resource.
DELETE	DELETE /cart/remove	Remove a resource (or, here, decrement one).

A few concrete design choices in this API, and the reasoning behind them:

- **Status codes carry meaning.** A successful creation returns `201 Created`, not a generic `200`. A missing resource returns `404`. An authenticated-but-not-allowed request returns `403`, while a not-authenticated-at-all request returns `401` — these are different failure modes and a well-behaved API distinguishes them.
- **Route ordering matters when paths overlap.** In `app/api/routers/books.py`, static routes like `/books/top-sellers` are registered *before* the catch-all `/books/{isbn}`. FastAPI matches path templates in registration order, so if the catch-all came first, a request for `/books/top-sellers` would get swallowed by the ISBN-lookup route with `isbn="top-sellers"`.
- **Response shape is explicit, not "whatever the ORM object happens to have."** Every endpoint declares a Pydantic `response_model`. This means a password hash can never accidentally leak in a response — the schema simply doesn't include that field, no matter what's on the underlying database row.
- **Nesting related data avoids extra round-trips.** A wishlist item's response includes the full book (title, price, etc.), not just a bare `book_id` the frontend would then have to look up separately. Compare this to the cart, which already did this — the wishlist schema was changed to match once the inconsistency became obvious while building the frontend.

Why FastAPI specifically

FastAPI generates the interactive documentation at `/docs` directly from your Python type hints and Pydantic schemas — you get a fully working API explorer for free, with no separate documentation to keep in sync. It's also built on Starlette/ASGI, so it's async-native, though this project uses it synchronously (SQLAlchemy's classic sync API) since the added complexity of async DB drivers wasn't worth it at this scale.

5. Authentication and authorization — and the difference between them

These two words get used interchangeably in casual conversation, but they answer different questions, and this project treats them as genuinely separate concerns.

Authentication answers: *who are you?* A user proves their identity once, at login, by presenting a username and password.

Authorization answers: *are you allowed to do this specific thing?* Every single request after login has to be checked again — being a known, logged-in user doesn't automatically mean you're allowed to read someone else's cart.

How authentication works here (JWT)

1. A user submits credentials to `POST /auth/login`.
2. `users_service.authenticate_user` looks up the user and calls `verify_password`, which runs bcrypt's comparison — the stored value is a one-way hash, so this never "decrypts" anything, it re-hashes the login attempt and compares.
3. On success, `create_access_token` builds a JWT: a signed, tamper-evident token containing the username and an expiry, encoded with a secret key only the server knows (`SECRET_KEY`).
4. The frontend stores that token (in `localStorage`) and attaches it as `Authorization: Bearer <token>` on every subsequent request that needs auth.
5. On each such request, `get_current_user` decodes and verifies the token's signature, pulls out the username, and loads the matching `User` row fresh from the database. If the signature doesn't check out or the token's expired, it's a `401`.

Why a JWT instead of a server-side session

A JWT is self-contained and signed — the server doesn't need to store anything to validate it (no session table, no server-side lookup on every request beyond the DB query to fetch the user). The tradeoff is that a JWT can't be "revoked" the way deleting a session row can; it's simply valid until it expires. For a project this size that tradeoff is a non-issue; for a system needing instant revocation (e.g. immediately locking out a compromised account), you'd add a server-side denylist or move to short-lived tokens with refresh tokens.

How authorization works here

Two layers, both real:

1. **Role-based:** `get_current_admin_user` is the same as `get_current_user`, plus a check that `current_user.is_admin` is true. Used on catalog-mutating routes.
2. **Ownership-based:** most routes aren't about roles at all — they're about whether *this specific resource* belongs to *this specific caller*. A helper like `_ensure_self` in `cart.py` and `users.py` compares the resource's owner id against `current_user.id` and raises 403 on a mismatch. The wishlist router does the analogous check by loading the wishlist and comparing `wishlist.user_id`.

The bug this project fixed

Before this pass, none of the cart/wishlist/profile endpoints checked ownership at all — any authenticated user could read or empty *any other user's* cart just by changing a number in the URL. This is one of the most common real-world API vulnerabilities (sometimes called IDOR — Insecure Direct Object Reference). It's an easy thing to miss because the endpoint "works" in every manual test where you're only ever testing as yourself.

Granting admin: a deliberate non-feature

There is no HTTP endpoint anywhere in this API that grants admin rights to a user. That absence is intentional, not an oversight. Promoting an account to admin is a privileged, infrequent action — exposing it over HTTP, even behind an "admin-only" check, adds attack surface for a feature that's barely ever used and is unusually dangerous if abused. Instead, `scripts/make_admin.py` is a small command-line script that talks to the database directly. This mirrors how many real systems handle "break glass" operations: through ops-only tooling, not the public API.

6. Databases, ORMs, and migrations

What an ORM is doing for you

SQLAlchemy is an **Object-Relational Mapper**: it lets you work with Python classes and objects (`User`, `Book`) instead of writing raw SQL for every query, and it translates between the two. A model like:

```
class Book(Base):
    __tablename__ = "books"
    id = Column(Integer, primary_key=True)
    title = Column(String(255), nullable=False)
    price = Column(DECIMAL(10, 2), nullable=False)
    author_id = Column(Integer, ForeignKey("authors.id"))
    author = relationship("Author", back_populates="books")
```

declares both the table's shape (columns, types, constraints) and its relationships to other tables. `db.query(Book).filter(Book.genre == "Systems").all()` becomes a real `SELECT` statement, and `book.author.first_name` triggers a second query (or a join, depending on how it's loaded) instead of you writing that join by hand.

Why migrations instead of just creating tables

The original version of this project called `Base.metadata.create_all()` on startup — it looked at the current model definitions and created any tables that didn't exist yet. That works for a brand new, empty database, but it has a fatal limitation: it never *changes* an existing table. Rename a column, add a constraint, and `create_all()` does nothing — the running database silently drifts out of sync with the code.

Alembic solves this by keeping a versioned history of schema changes as Python scripts (`alembic/versions/*.py`), each with an `upgrade()` and a `downgrade()`. Instead of "make the schema match the models" once, you get:

```
# after changing a model:
alembic revision --autogenerate -m "add is_admin to users"
alembic upgrade head
|
# to roll back the most recent change:
alembic downgrade -1
```

Every environment — your laptop, a teammate's laptop, CI, production — runs the same ordered list of migrations and ends up with an identical schema. This project's `alembic/env.py` is wired to import `app.database.DATABASE_URL` directly, so the same migrations apply whether you're pointed at a local SQLite file or a deployed MySQL instance.

A trick worth stealing

This project verifies migrations by generating and applying them against a disposable SQLite database first (`DATABASE_URL=sqlite:///./_check.db alembic upgrade head`), even though production runs MySQL. It's not a perfect substitute for testing against the real engine, but it catches ordering bugs (a foreign key referencing a table that hasn't been created yet) and typos without needing a MySQL server running locally. The GitHub Actions workflow does the same thing on every push.

7. Automated testing — why, and how this project does it

The 27 tests in `tests/` exist to answer one question automatically, every time the code changes: *did I just break something that used to work?* Without tests, that question only gets answered by manually clicking through the app — which is slow, easy to skip under deadline pressure, and never covers every edge case (particularly security-relevant ones, like "can user B read user A's cart").

The key trick: an isolated test database

`tests/conftest.py` overrides the app's `get_db` dependency with one pointed at an in-memory SQLite database, rebuilt fresh before every single test:

```
@pytest.fixture(autouse=True)
def _reset_db():
    Base.metadata.create_all(bind=engine)
    yield
    Base.metadata.drop_all(bind=engine)
app.dependency_overrides[get_db] = override_get_db
```

This means tests never touch your real development database, never leave behind test users or fake books, and can run in any order without one test's leftover data breaking another. It also means the test suite needs zero external setup — no MySQL server, no seed data — which is exactly why it can run unmodified in GitHub Actions.

What's actually being tested

Beyond basic "does this endpoint return what I expect," a meaningful chunk of the suite exists specifically to prove the authorization work actually holds:

```
def test_cannot_access_another_users_cart(client, admin_headers, auth_headers, make_user):
    bob_headers = make_user("bob")
    user_id = _my_id(client, auth_headers)
    resp = client.get(f"/cart/items/{user_id}", headers=bob_headers)
    assert resp.status_code == 403
```

This is the difference between "the app works" and "the app is actually secure" — a test suite that only checks the happy path would never have caught the IDOR bug mentioned in Section 5.

8. Docker, from zero: what it is and how this project uses it

The problem Docker solves

"It works on my machine" is a real and common failure mode: your laptop has Python 3.13, a specific version of MySQL, certain environment variables set, and a dozen other small facts about its state that you've forgotten are load-bearing. Someone else's machine — or your production server — doesn't have all of that, and the app breaks in ways that are maddening to debug because the *code* didn't change.

A **container** packages your application together with everything it needs to run — the exact Python version, every dependency, the OS-level libraries — into a single unit that behaves identically no matter where it runs. It's not a full virtual machine (which would emulate entire hardware and run a full separate OS kernel); it's a lighter-weight isolation built on features of the host's own Linux kernel, which is why containers start in under a second and share the host's kernel instead of booting their own.

The core building blocks

Term	What it means
Image	A read-only snapshot: your app's code plus every dependency and OS package it needs, built in layers. Think of it as a frozen, portable installation.
Container	A running instance of an image. You can start several containers from the same image at once, each isolated from the others.
Dockerfile	A text file of instructions describing how to build an image, step by step (install dependencies, copy code in, define the startup command).
docker-compose.yml	Describes a set of containers that need to run together — in this project, the API and a MySQL database — and how they're networked and configured, so <code>docker compose up</code> starts the whole stack with one command.
Registry	A place images are stored and pulled from (Docker Hub is the default public one). Deploy platforms like Railway build your image and run it without you ever pushing to a registry yourself.

This project's Dockerfile, annotated

```
FROM python:3.13-slim          # start from a minimal official Python image
|
|
WORKDIR /app                   # everything below happens in /app inside the container
|
|
RUN apt-get update \           # OS-level packages PyMySQL's C extension needs to build
  && apt-get install -y --no-install-recommends default-libmysqlclient-dev build-essential \
  && rm -rf /var/lib/apt/lists/*
|
|
COPY requirements.txt .         # copied separately from the rest of the code on purpose --
RUN pip install --no-cache-dir -r requirements.txt # see callout below
|
|
COPY alembic.ini
COPY alembic ./alembic
COPY app ./app
|
|
EXPOSE 8000                     # documents which port the app listens on
|
|
CMD ["sh", "-c", "alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port 8000"]
```

Why requirements.txt is copied before the rest of the code

Docker caches each instruction's result as a layer, and reuses a cached layer if nothing that affects it has changed. Copying `requirements.txt` and running `pip install` *before* copying the application code means that as long as your dependencies haven't changed, Docker skips reinstalling them on every rebuild — only your actual code changes trigger a fast final layer. Copy everything in one step and every single code edit forces a full dependency reinstall.

What docker-compose.yml adds

A single container is one process. This app needs two things running together — the API and a database — and needs the API to wait until the database is actually ready to accept connections before it tries to run migrations against it. `docker-compose.yml` declares both services, a named volume so the database's data survives a container restart, and a healthcheck the API service waits on:

```
services:
  db:
    image: mysql:8.4
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", ...]
      interval: 5s
```

```
retries: 10
backend:
  build: .
  depends_on:
    db:
      condition: service_healthy # wait for the healthcheck, not just "container started"
  environment:
    DATABASE_URL: mysql+pymysql://root:...@db:3306/bibliotech
```

Notice the hostname in that `DATABASE_URL` is `db`, not `localhost` — Compose creates a private network where each service can reach the others by its service name.

Commands you'll actually use

Command	What it does
<code>docker compose up --build</code>	Build images (if needed) and start every service, attached to your terminal so you see the logs.
<code>docker compose up -d</code>	Same, but detached — runs in the background.
<code>docker compose down</code>	Stop and remove the containers (the named volume, and therefore your data, survives unless you also pass <code>-v</code>).
<code>docker compose logs -f backend</code>	Stream logs from just the backend service.
<code>docker ps</code>	List running containers.
<code>docker exec -it <container> sh</code>	Open a shell inside a running container — useful for poking around when something's not behaving as expected.

Honest note

Docker wasn't installed in the environment this project was actually built and tested in, so `docker compose up` for this specific repo was written carefully and reviewed, but never run end-to-end during development. Everything else in this guide was verified directly. If you hit an issue running it, that's the first place to look.

9. Continuous integration (CI)

CI means: every time code is pushed, a server automatically runs your checks (tests, builds, linters) and tells you whether it's still healthy — without anyone having to remember to run them by hand. This project's pipeline ([.github/workflows/ci.yml](#), running on GitHub Actions) does three things on every push or pull request to `main`:

1. Installs backend dependencies and runs `pytest`.
2. Validates that Alembic migrations apply cleanly against a scratch SQLite database.
3. Installs frontend dependencies and runs `npm run build`, catching any TypeScript error that would otherwise only surface when someone tried to deploy.

The value isn't really about catching mistakes *you'd* make — it's a safety net for the version of the project where more than one person is contributing, or where enough time has passed that you've forgotten a subtlety you used to hold in your head. A green checkmark on a pull request is a fast, objective answer to "is this safe to merge," instead of a manual re-verification every time.

10. The frontend: how it talks to the backend

The frontend is a React + TypeScript single-page app built with Vite, using React Router for client-side navigation. It never touches the database directly — every piece of data on screen came from a `fetch` call to the API described in the earlier sections.

The shape of the frontend's own layering

Folder	Job
<code>api/</code>	One typed module per backend resource (<code>books.ts</code> , <code>cart.ts</code> , ...), all built on a shared <code>client.ts</code> that wraps <code>fetch</code> : attaches the JWT, serializes the body, and turns a non-2xx response into a typed <code>ApiError</code> with the backend's actual error message.
<code>context/AuthContext</code>	The one source of truth for "who's logged in right now," exposed to the whole app via React Context so any component can read <code>user</code> or call <code>login/logout</code> without prop drilling.
<code>components/</code>	Reusable pieces: <code>NavBar</code> , <code>BookCard</code> , <code>StarRating</code> , and <code>ProtectedRoute</code> (a route guard that redirects to <code>/login</code> if logged out, or home if an <code>adminOnly</code> page is hit by a non-admin).
<code>pages/</code>	One file per screen, composing the above into the actual UI.

Where the JWT actually lives

On login, the backend's token is stored in `localStorage` under a fixed key. Every subsequent authenticated request reads it back out and sends it as an `Authorization: Bearer <token>` header — this is the entire mechanism that makes the frontend "stay logged in." On page load, `AuthContext` checks for a stored token and, if one exists, calls `GET /auth/me` to restore the session; this is what survives a page refresh instead of bouncing straight to the login screen.

CORS: the other half of the auth story

The frontend and backend run on different origins in development (`localhost:5173` vs `localhost:8000`) — and different origins entirely once deployed. Browsers block cross-origin requests by default (Cross-Origin Resource Sharing). `app/main.py` explicitly allows the frontend's origin via `CORSMiddleware`,

configured from the `CORS_ORIGINS` environment variable so it can be updated per-environment without a code change.

11. Mistakes made and fixed along the way — real lessons

Every one of these actually happened during this project's development. They're included because the lesson from a real bug sticks better than a hypothetical one.

1. A UTF-16 requirements.txt

`requirements.txt` had been saved with a UTF-16 encoding (visible as garbled spaced-out characters when read as plain text) and was missing half its actual dependencies. A fresh `pip install -r requirements.txt` would silently fail to install `fastapi` itself. **Lesson:** a file that "looks right" in your editor can still be wrong at the byte level — this is exactly the kind of bug that only shows up on someone else's machine.

2. A crashing endpoint

`book_service.create_book` built a `BookCreate` — a Pydantic validation schema, not a database model — and passed it to `db.add()`. SQLAlchemy has no idea what to do with an object that isn't one of its mapped classes, so every single call to this endpoint raised an exception. **Lesson:** a schema and a model can look superficially similar (same field names) while being completely different kinds of object — mixing them up is an easy, silent mistake if you're not careful about which layer you're in.

3. A wrong-column bug

`remove_item_from_cart` filtered `ShoppingCart.id == user_id` instead of `ShoppingCart.user_id == user_id` — a one-character difference that only breaks for the (very common) case where a cart's id happens to differ from its owner's user id. **Lesson:** bugs like this hide well in manual testing, because if you're the first or only user, your cart id and user id often line up by coincidence.

4. Writing a .env file with PowerShell's echo >

While setting up local development, running `echo DATABASE_URL=... > .env` in PowerShell wrote the file in UTF-16 — Windows PowerShell's default redirection encoding — which `python-dotenv` can't parse, causing a `UnicodeDecodeError` on startup. **Lesson:** the exact same encoding trap can bite you twice in one project if you don't recognize the pattern the second time; it's now called out explicitly in this project's README.

5. An IDOR vulnerability

Covered in Section 5 — no endpoint checked that the caller actually owned the resource they were accessing. **Lesson:** "does it work" and "is it secure" are different questions, and only one of them is

obvious from clicking around the app yourself.

12. A playbook for building your own version of this

If you're starting a similar project from scratch, here's a condensed, opinionated order of operations, distilled from the timeline in Section 2:

1. **Get the domain model right first.** What are the real entities (users, products, orders...) and how do they relate? Sketch this before writing any code — schema changes get more expensive the more code depends on them.
2. **Build the thinnest possible vertical slice** — one resource, one CRUD endpoint, working end to end — before building the rest breadth-first. This catches structural problems (framework setup, database connection, deployment target) early, when they're cheap to fix.
3. **Add authentication before you add more than one or two protected endpoints.** Retrofitting auth onto ten already-built endpoints means touching all ten; adding it after one or two means the pattern is established once.
4. **Write authorization checks as you write each endpoint, not as a separate later pass.** This project did do it as a separate pass, and it worked — but only because the codebase was still small. On anything larger, "add auth later" reliably means "some endpoints get missed."
5. **Add migrations as soon as your schema stabilizes even a little.** Every day you delay is another schema change that's harder to express as a clean migration later.
6. **Write tests for the behavior you're least confident about first** — usually authorization boundaries and anything with tricky business rules (max-3-wishlists, create-or-update semantics). Tests for simple CRUD are nice to have; tests for "can user B see user A's data" are the ones that actually catch real bugs.
7. **Containerize once the app is stable enough that you're not fighting the Dockerfile and the application code at the same time.**
8. **Build the frontend against a real, running API** — not mocked data. This is what surfaces API design gaps (like the missing catalog search endpoint here) while they're still cheap to add.
9. **Document last, but don't skip it.** A README that explains real, verified setup steps is worth more than one written speculatively before you know what actually works.

The meta-lesson

Almost every "hardening" step in this guide is really the same move applied to a different layer: find the place where the code currently trusts something it shouldn't (a client input, an unhashed password, an untested assumption, an unversioned schema, a manually-run checklist) and replace that trust with a mechanism that

enforces it automatically. That single habit — *don't trust what you can verify* — is most of what separates a working prototype from production-shaped code.

13. Appendix: commands, tech stack, folder structure

Tech stack

Layer	Tools
API	FastAPI, Pydantic v2, Uvicorn
ORM / DB	SQLAlchemy 2.0, MySQL, Alembic
Auth	JWT (python-jose), bcrypt (passlib)
Testing	pytest, httpx, SQLite (test-only)
Frontend	React, TypeScript, Vite, React Router
Infra	Docker Compose, GitHub Actions

Everyday commands

```
# Backend
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
alembic upgrade head
uvicorn app.main:app --reload
pytest

# Frontend
cd frontend && npm install && npm run dev
npm run build

# Docker
docker compose up --build
docker compose down

# Grant admin (no HTTP endpoint for this -- see Section 5)
python scripts/make_admin.py yourusername

# New migration after changing a model
alembic revision --autogenerate -m "describe the change"
```

```
alembic upgrade head
```

Folder structure

```
app/  
  main.py          FastAPI app, CORS, router registration  
  database.py      engine/session setup  
  core/security.py password hashing, JWT encode/decode  
  api/  
    deps.py        get_db, get_current_user, get_current_admin_user  
    routers/       HTTP layer -- one file per resource  
    services/      business logic + DB queries  
    models/        SQLAlchemy ORM classes  
    schemas/       Pydantic request/response shapes  
alembic/           versioned schema migrations  
tests/            pytest suite (isolated SQLite per test)  
scripts/make_admin.py CLI-only admin grant (no HTTP equivalent)  
frontend/src/  
  api/            typed fetch client + one module per resource  
  context/        AuthContext (JWT + current user)  
  components/     NavBar, BookCard, StarRating, ProtectedRoute  
  pages/          Catalog, book detail, cart, wishlist, auth, profile, admin
```

This guide was generated to accompany the BiblioTech repository. For the most current setup instructions, always defer to the README in the repository itself — this document reflects the state of the project as of its generation date.